

Bypass Certificate Pinning

Privacy Analysis of Augmented Reality Glasses

1 Objective

The goal is to bypass the certificate pinning mechanism of the Meta View app to be able to use all functionalities like asking for the weather.

2 Methods

I tried different approaches, from modifying the APK to using runtime debugging and hooking frameworks. These efforts either tried to disable the certificate pinning checks or dynamically bypassing them during app execution.

2.1 Objection + Frida

Objection is a mobile exploration tool built on Frida, which supports bypassing certificate pinning dynamically without modifying the APK.

General steps:

1. Install Objection and Frida
2. Run the app on the phone and launch Frida-server on the phone

```
adb push frida-server /data/local/tmp/  
adb shell chmod 755 /data/local/tmp/frida-server  
adb shell /data/local/tmp/frida-server &
```

3. Explore the app using the command:

```
objection -g com.facebook.stella explore
```

4. Use the built-in command to disable certificate pinning:

```
android sslpinning disable
```

Problems:

- After running the objection explore command, it would indicate that it is using the phone but then it freezes, which prevents further interaction. Have to kill process to stop the exploration. This could be due to the restrictions on the phone.
- Repeated errors such as Address already in use occurred when attempting to start Frida. This was solved by verifying then killing the conflicting processes:

```
adb shell ps | grep frida  
adb kill <processID>
```

- Errors encountered during Frida-server execution due to SELinux policies and non-rooted device restrictions. The error looks like:

```
Unable to load SELinux policy from the kernel: Failed to open file  
?/sys/fs/selinux/policy?: Permission denied
```

2.2 APK Modification with Apktool

Decompile the APK, modify the code to disable certificate pinning, and recompile and resign the APK.

General steps:

1. Decompile the apk using apktool with the command:

```
apktool d -f base.apk -o base-decompiled
```

2. Search for methods related to certificate verifications, including 'checkServerTrusted', 'CertificatePinner', and 'HostnameVerifier' in the decompiled files.

3. Edit the methods to either return immediately or to bypass the certificate checks.

4. Rebuild the apk using the command:

```
apktool b base-decompiled -o base-rebuilt.apk
```

5. Sign the apk using the command:

```
java -jar uber-apk-signer.jar --apks rebuilt.apk
```

6. Install the edited apk on the phone:

```
adb install rebuilt.apk
```

Problems:

- There were errors due to invalid XML values in styles.xml and other hundreds of resource files. For example, the file would have:

```
<item name="APKTOOL_DUMMYVAL_0x7f040043">false</item>
```

I tried to solve this by replacing them with mostly 'null' or proper values.

- There were errors when compiling the resources. The error looked like:

```
invalid resource directory name: /res navigation
```

Note that this directory does not exist for me, so I was not able to solve this compiling issue. I tried to avoid the decompilation of the resource entirely by using the '-no-res' flag, but that did not work.

- apk did not rebuild properly. I tried using '-use-aapt2' to attempt rebuilding the apk but faced additional errors related to resource compilation and malformed XML files.

2.3 Runtime Hooking with Frida

Hook into runtime methods like 'checkServerTrusted' to bypass certificate pinning during execution.

General steps:

1. Write a custom Frida script to hook into the method.
2. Inject the script using the command:

```
frida -U -n com.facebook.stella -e <script.js>
```

Problems:

- Non-rooted device restrictions prevented effective hooking.
- Frida-server issues due to SELinux and app protection mechanisms. The error looks like:

```
Unable to load SELinux policy from the kernel: Failed to open file  
?/sys/fs/selinux/policy?: Permission denied
```

2.4 APK Patching with apk-mitm

apk-mitm is a tool that automatically removes certificate pinning from Android APK files.

General steps:

1. Install apk-mitm
2. Run apk-mitm which does general SSL pinning bypass by applying standard patches to the apk using the command:

```
apk-mitm base.apk
```

Or, to specify the mitmproxy certificate into the app's network security configuration in addition to patching SSL pinning, meaning to use apk-mitm to modify the app and inject the mitmproxy certificate:

```
apk-mitm --certificate ~/Downloads/mitmproxy.pem ~/Downloads/base.apk
```

Problems:

- The bulding of the resources fails due to malformed XML files. This is how it should look:

```
$ apk-mitm example.apk
✓ Decoding APK file
✓ Modifying app manifest
✓ Replacing network security config
✓ Disabling certificate pinning
✓ Encoding patched APK file
✓ Signing patched APK file
```

And this is what I see:

```
✓ Checking prerequisites
✓ Decoding APK file
✓ Applying patches
} Encoding patched APK file
↓ Encoding using AAPT2 [skipped]
  → Failed, falling back to AAPT...
✓ Encoding using AAPT [fallback]
  → I: Building resources...
    Signing patched APK file
...
Failed! An error occurred:
...
I:Building resources
...
W: <tmp_dir>/decode/res/layout/APKTOOL_DUMMYVAL_0x7f0e0025.xml:7:
error: not well-formed (invalid token).
...
```

2.5 Android SSL Unpinning

The Android SSL modifies the apk to disable SSL pinning dynamically by injecting Frida and HTTP Toolkit's unpinning scripts. General steps:

1. Install android unpinner
2. Run the command:

```
android-unpinner all base.apk
```

Problems:

- The process aborted. The following is what I see:

```
[18:33:59] Target: com.facebook.stella
[18:33:59] Reusing existing file: /home/michelle/Downloads/base.unpinned.apk
[18:33:59] Installing com.facebook.stella...
[18:34:07] Detect architecture...
[18:34:07] Copying matching gadget: frida-gadget-16.0.10-android-arm64.so...
[18:34:08] Copying builtin Frida scripts to /data/local/tmp/android-unpinner...
[18:34:08] Active frida scripts: ['hide-debugger.js', 'httptoolkit-unpinner.js']
[18:34:08] Start app (suspended)...
[18:34:08] Obtain process id...
[18:34:10] Establish Java Debug Wire Protocol Connection over ADB...
Aborted.
```

The issue appears to stem from JDWP connection which could be due to the phone not allowing adb debugging for the app.